

JOB SEARCH MANAGEMENT SYSTEM

C# Architecture, Domain Model, Gmail Ingestion Pipeline & MVP
Backlog

Product thesis

A job tracker you do not have to maintain.

Architecture blueprint for a .NET / Blazor implementation
August 2026

1. Executive Summary

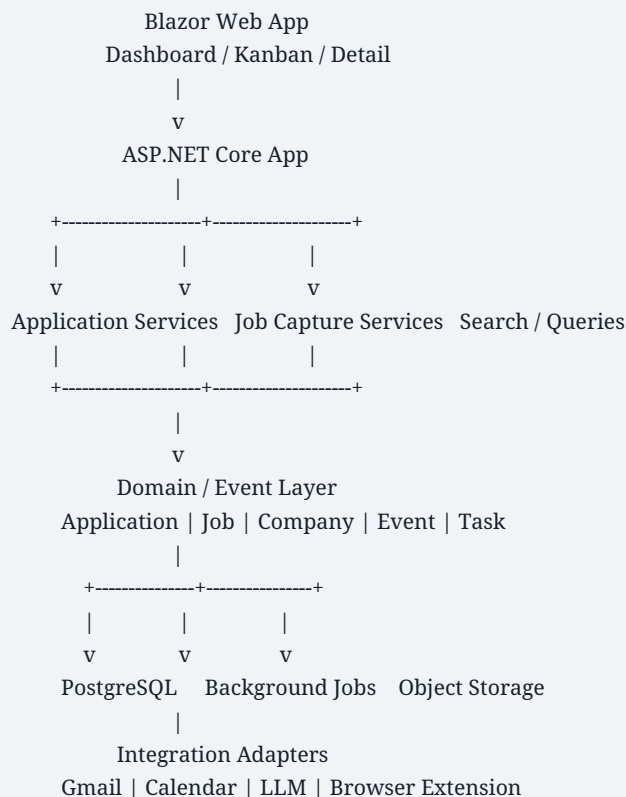
Build the product as a modular monolith. Use ASP.NET Core and Blazor for the application, EF Core and PostgreSQL for persistence, a lightweight worker for background synchronization, Gmail as an event source, and an LLM only for classification and structured extraction. Keep business state transitions deterministic and auditable.

Core rule: External systems generate facts. AI proposes structured events. The domain model owns truth, state transitions, audit history, and reminders.

The system should optimize for one outcome: eliminating manual job-search bookkeeping. A user should save a job once, apply, and then allow email and calendar activity to maintain the application timeline automatically.

- Primary daily experience: an Attention view showing what needs action now.
- Primary system of record: JobApplication plus immutable JobEvent history.
- Primary automation source: Gmail; Calendar follows in a later slice.
- Primary AI role: classify, extract, summarize, and propose - never directly mutate domain state.
- Primary capture mechanism: browser extension that stores the original job description before processing.

2. High-Level Architecture



Recommended solution structure

```
JobHunt.sln

src/
  JobHunt.Web      Blazor UI, API endpoints, authentication
  JobHunt.Application  Commands, queries, services, event handlers
  JobHunt.Domain   Entities, value objects, domain rules
  JobHunt.Infrastructure  EF Core, Gmail, Calendar, LLM, storage
  JobHunt.Worker   Sync, email processing, reminders

tests/
  JobHunt.Domain.Tests
  JobHunt.Application.Tests
  JobHunt.IntegrationTests
```

Start simple: The worker can run inside the web process for the prototype. Split it only when the system proves useful. A PostgreSQL-backed inbox/outbox table is sufficient; do not introduce a message broker yet.

3. Core Domain Model

Make JobApplication the primary aggregate for active job-search state, but keep JobPosting separate. This allows the system to retain interesting jobs that never become applications.

```
Company
|-- Contacts
|-- JobPostings
|
+-- JobApplication
    |-- JobEvents
    |-- ApplicationTasks
    |-- ApplicationDocuments
    |-- ApplicationContacts
    |-- Interviews
    +-- ExternalMessages
```

JobPosting

```
public class JobPosting
{
    public Guid Id { get; set; }
    public Guid CompanyId { get; set; }
    public string Title { get; set; } = "";
    public string? Url { get; set; }
    public string? Location { get; set; }
    public decimal? SalaryMin { get; set; }
```

```

    public decimal? SalaryMax { get; set; }
    public string? DescriptionText { get; set; }
    public string? DescriptionHtml { get; set; }
    public DateTimeOffset CapturedAt { get; set; }
    public JobSource Source { get; set; }
}

```

JobApplication

```

public class JobApplication
{
    public Guid Id { get; set; }
    public Guid JobPostingId { get; set; }
    public ApplicationStatus Status { get; private set; }
    public DateTimeOffset? AppliedAt { get; private set; }
    public DateTimeOffset? ClosedAt { get; private set; }
    public Guid? ResumeDocumentId { get; set; }
    public Guid? CoverLetterDocumentId { get; set; }
    public ICollection<JobEvent> Events { get; set; } = [];
    public ICollection<ApplicationTask> Tasks { get; set; } = [];
}

```

Application status should stay coarse

```

public enum ApplicationStatus
{
    Interested,
    Preparing,
    Applied,
    RecruiterScreen,
    Interviewing,
    Offer,
    Rejected,
    Withdrawn,
    Closed
}

```

Avoid making statuses such as `SecondTechnicalInterview` or `HiringManagerRound`. Those are timeline events, not durable workflow states.

4. JobEvent: The Heart of the System

```

public class JobEvent
{
    public Guid Id { get; set; }
    public Guid JobApplicationId { get; set; }
    public JobEventType Type { get; set; }
}

```

```

    public DateTimeOffset OccurredAt { get; set; }
    public EventSource Source { get; set; }
    public string? Summary { get; set; }
    public Guid? SourceMessageId { get; set; }
    public decimal? Confidence { get; set; }
    public string? MetadataJson { get; set; }
}

```

Event family	Examples
Application	JobSaved, ApplicationStarted, ApplicationSubmitted, ApplicationConfirmed
Recruiting	RecruiterContacted, PhoneScreenRequested, PhoneScreenScheduled, PhoneScreenCompleted
Interviewing	InterviewRequested, InterviewScheduled, InterviewCompleted
Follow-up	FollowUpNeeded, FollowUpSent
Terminal	OfferReceived, Rejected, Withdrawn
Human context	NoteAdded

A timeline becomes the authoritative audit trail:

```

Aug 03  Job Saved
Aug 04  Application Submitted
Aug 04  Application Confirmed
Aug 08  Recruiter Contacted
Aug 10  Phone Screen Scheduled
Aug 13  Phone Screen Completed
Aug 15  Interview Requested
Aug 21  Interview Scheduled

```

5. Deterministic State Transitions

AI classification should never directly set an application status. Instead, the AI proposes a `JobEventType`. Domain rules translate that event into state.

```

public void Apply(JobEvent jobEvent)
{
    switch (jobEvent.Type)
    {
        case JobEventType.ApplicationSubmitted:
        case JobEventType.ApplicationConfirmed:
            Status = ApplicationStatus.Applied;
            break;

        case JobEventType.PhoneScreenRequested:
        case JobEventType.PhoneScreenScheduled:

```

```

        Status = ApplicationStatus.RecruiterScreen;
        break;

    case JobEventType.InterviewRequested:
    case JobEventType.InterviewScheduled:
        Status = ApplicationStatus.Interviewing;
        break;

    case JobEventType.OfferReceived:
        Status = ApplicationStatus.Offer;
        break;

    case JobEventType.Rejected:
        Status = ApplicationStatus.Rejected;
        ClosedAt = jobEvent.OccurredAt;
        break;
    }
}

```

Why this matters: The model can be wrong without corrupting business logic. You can review, reject, replay, or reclassify an extracted event while preserving a complete audit trail.

6. Gmail Ingestion Pipeline

```

Gmail
|
v
Sync Messages
|
v
Deduplicate
|
v
Cheap relevance filter -----> irrelevant -> ignore
|
v
Identify / match application
|
v
LLM structured extraction
|
v
JobEventCandidate
|
+---- low confidence -----> Review Queue
|
v
Validate / approve
|

```

```
v
Create JobEvent
|
v
Apply domain rules
|
v
Update application + create task if needed
```

Stage A - Is this job-related?

Use deterministic filtering first. Known ATS senders and obvious subject/body patterns can avoid an unnecessary model call.

Known systems / signals:

greenhouse
lever
workday
icims
ashby
smartrecruiters

Common phrases:

"thank you for applying"
"application received"
"interview"
"recruiter"
"candidate"

Stage B - Which application does it belong to?

Resolve entities using sender domain, company name, job title, ATS body content, and the user's currently open applications. If multiple applications plausibly match, do not guess - route the candidate to review.

Important simplification: Once the user saves or applies to a job, the system already knows the company, role, URL, and likely domains. Matching an email against known applications is much easier than asking a model to infer everything from scratch.

7. LLM Contract and Candidate Staging

```
public record EmailAnalysisResult(
    bool IsJobRelated,
    string? Company,
    string? JobTitle,
    JobEventType? EventType,
    string? ContactName,
    string? ContactEmail,
```

```
string? Summary,  
string? NextAction,  
DateTimeOffset? ActionDueAt,  
decimal Confidence);
```

Keep the prompt narrow and structured: classify the message, extract supported facts, suggest a next action, and return the schema. Do not ask the model to act as a free-form agent during ingestion.

Analyze this email as an event in a job application process.
Do not make assumptions not supported by the email.
Determine whether it is job-related, company, position, event type,
people mentioned, requested action, relevant dates, and confidence.
Return only the structured schema.

JobEventCandidate

```
public class JobEventCandidate  
{  
    public Guid Id { get; set; }  
    public Guid ExternalMessageId { get; set; }  
    public Guid? ProposedApplicationId { get; set; }  
    public JobEventType ProposedType { get; set; }  
    public decimal Confidence { get; set; }  
    public string ExtractionJson { get; set; } = "";  
    public CandidateStatus Status { get; set; }  
}
```

Confidence / type	Initial behavior
≥ 0.95 , routine event	May auto-apply after trust is established
0.70 - 0.95	Review queue
< 0.70	Ignore or review depending on signal
Rejected / OfferReceived / Withdrawn	Require manual confirmation initially

8. Gmail Synchronization Strategy

For a single-user prototype, polling is enough. Run a background job every few minutes, fetch messages newer than the last sync point, deduplicate by provider message ID, and process only unseen messages.

```
Every few minutes  
|  
v  
Fetch messages newer than LastSync  
|  
v
```



```

Persist ProviderMessageId / ProviderThreadId
|
v
Queue unprocessed messages
|
v
Analyze -> candidate -> event

```

```

public class GmailSyncState
{
    public string AccountId { get; set; } = "";
    public ulong? HistoryId { get; set; }
    public DateTimeOffset LastSyncAt { get; set; }
}

```

Optimize with Gmail history-based incremental sync only after the basic pipeline is reliable.

9. Design for Multiple Message Sources

Even if Gmail is the only V1 provider, keep Gmail out of the domain model. Normalize provider messages into an ExternalMessage abstraction.

```

Gmail -----+
|
Outlook -----+----> ExternalMessage ----> MessageAnalyzer ----> JobEventCandidate
|
Manual -----+

```

```

public enum ExternalSource
{
    Gmail,
    Outlook,
    LinkedIn,
    Manual
}

```

Small abstraction, large payoff: Outlook becomes another adapter rather than a rewrite of the ingestion pipeline.

10. Job Description Capture

Use a browser extension instead of server-side scraping. Capture the raw page before any AI processing so the system always retains the exact posting the user saw.

```

Browser Extension
|
v
Read current page
|
v
Capture URL + HTML + normalized text + timestamp
|
v
Send to JobHunt API
|
v
AI extracts company / title / salary / location
|
v
User confirms -> Save JobPosting

```

- Store original URL.
- Store captured HTML.
- Store normalized plain text.
- Store extraction result separately from raw source.
- Calculate a SHA-256 hash of normalized JD text to detect later changes.

11. Documents, Contacts, and Tasks

Documents

```

ApplicationDocument
  Id
  JobApplicationId
  DocumentType
  FileName
  StorageKey
  CreatedAt

DocumentType:
  Resume | CoverLetter | JobDescription | InterviewNotes | RecruiterNotes | Other

```

This preserves one of the most useful pieces of history: exactly which resume and cover letter were submitted for a particular role.

Contacts

```

Contact
  Id
  CompanyId
  Name
  Title
  Email

```

LinkedInUrl
Notes

ApplicationContact
ApplicationId
ContactId
Role

Contact roles can include recruiter, hiring manager, interviewer, referral, or other. AI can propose contacts from email signatures.

Follow-up tasks

ApplicationTask
Id
JobApplicationId
Description
DueAt
Status
Source

Examples: follow up with recruiter, send thank-you, prepare system-design examples, or respond with interview availability.

12. Daily User Experience

Kanban dashboard

INTERESTED	APPLIED	INTERVIEWING	OFFER
Acme	Contoso	PetDesk	Foo
Rocket Lab	Fabrikam	Widgets Inc.	
Some Startup			

Cards should show only actionable context: role, company, current state, last activity age, next action, and relevant contact.

Application detail

PetDesk - VP Engineering INTERVIEWING

Next action
Follow up Aug 21

Timeline
Aug 11 Interview with Michael
Aug 09 Interview scheduled
Aug 07 Recruiter contacted

Aug 05 Applied

[Contacts](#) | [Notes](#) | [Documents](#) | [Job Posting](#)

Attention screen

GOOD MORNING

3 items need attention

PetDesk

No communication for 9 days

-> Consider following up

Acme

Interview tomorrow at 10:00

-> Prepare for interview

Widgets Inc.

Recruiter asked for availability yesterday

-> Reply required

JOB SEARCH

24 active | 5 interviewing | 13 applied | 6 waiting

Product differentiator: The system should answer "What do I need to do?" rather than merely displaying a database of applications.

13. AI Boundary for MVP

Capability	AI?	Notes
Determine whether email is job-related	Sometimes	Use rules first, model for ambiguity
Extract company / role	Yes	Structured extraction
Classify email event	Yes	Produces candidate event
Summarize email	Yes	For timeline and attention view
Extract contact	Yes	Proposed contact record
Suggest next action	Yes	Create candidate task
Application state transition	No	Deterministic domain rules
Database mutation	No	Application services only
Job description extraction	Yes	Raw source still retained

Job/company matching	Later	Useful after core workflow works
Resume rewriting	Not initially	Avoid scope creep
Interview coaching	V2	High-value follow-on

14. MVP Backlog

Priority	Story	Reason
P0	Create/edit company	Foundation
P0	Save job posting	Foundation
P0	Create application from posting	Foundation
P0	Application status	Foundation
P0	Application timeline	Core UX
P0	Add notes	Core UX
P0	Attach resume/document	Core UX
P0	Kanban dashboard	Daily interface
P0	Gmail OAuth	Automation foundation
P0	Gmail incremental sync	Automation
P0	Detect job-related emails	Automation
P0	AI structured extraction	Automation
P0	Match email to application	Automation
P0	Generate candidate event	Safety
P0	Review/approve candidate event	Safety
P0	Apply event to application state	Automation
P1	Automatic high-confidence events	Remove manual work
P1	Follow-up tasks	Daily usefulness
P1	Stale application detection	Daily usefulness
P1	Browser extension	Fast capture
P1	Capture complete JD	Knowledge retention
P1	Contact extraction	Interview usefulness
P1	Full-text search	Knowledge retrieval
P2	Calendar integration	Interview automation
P2	Interview records	Richer workflow
P2	AI application assistant	Intelligence layer
P2	Analytics	Optimize search

P2	JD/resume matching	Application preparation
P3	Job discovery	Separate problem
P3	Application autofill	Browser complexity
P3	Auto-apply	Probably never

15. First Usable Milestone

MVP 0.1 should prove the end-to-end automation loop with the smallest possible surface area.

1. Find a job.
2. Paste its URL and job description into the app.
3. Create the application.
4. See it on the dashboard.
5. Allow Gmail to synchronize.
6. Receive an application confirmation and have it added automatically to the timeline.
7. Receive recruiter outreach and have the contact plus event proposed automatically.
8. Receive an interview request and have the application move to Interviewing after validation.
9. Open the application weeks later and still have the original JD, submitted resume, correspondence context, notes, contacts, and timeline.

Success criterion: If this loop is reliable, the product is already useful. Everything else is refinement.

16. Recommended Build Sequence

Slice	Deliverable	Exit condition
1. Manual tracker	Companies, postings, applications, status, notes, documents, Kanban	Usable without integrations
2. Permanent JD capture	Browser/manual capture, raw source retention, extraction	Original posting is always recoverable
3. Gmail to timeline	OAuth, sync, filtering, matching, AI extraction, candidate review	Emails reliably create correct timeline candidates
4. Attention + tasks	Next actions, stale detection, due tasks, daily attention view	System tells user what needs action
5. Calendar	Interview event detection and scheduling context	Upcoming interviews appear automatically
6. Assistant	Read-only tools over applications, JDs, contacts, notes and events	Natural-language querying adds value without controlling truth

17. Add the Agent Later

Do not start with a general-purpose agent that can directly manipulate Gmail, Calendar, the database, and documents. Build deterministic application services first, then expose safe read/query operations as tools to an assistant.

Deterministic services first:

- GmailSyncService
- MessageAnalyzer
- ApplicationMatcher
- EventCandidateService
- ApplicationService
- ReminderService
- JobCaptureService

Later assistant tools:

- SearchApplications
- GetApplicationTimeline
- GetJobPosting
- GetContacts
- GetUpcomingTasks
- GetInterviewContext

Best first AI feature: Not chat. Make data entry disappear. Conversational intelligence becomes valuable after the system has accumulated trustworthy job-search memory.

18. Persistence, Observability, and Deployment

System tables worth adding early

- GmailSyncState
- OutboxEvent
- AIRequestLog

AIRequestLog should capture model, prompt version, token counts, estimated cost, latency, extraction result, application ID, and message ID. This makes classification failures explainable and reproducible.

Deployment choices

- Local prototype
- Docker Compose
 - app
 - postgres

Hosted later

- Azure App Service or Container Apps
- Azure Database for PostgreSQL

Azure Blob Storage
Google OAuth
LLM API

For personal use, local hosting is entirely reasonable during development. Deploy only when remote access and always-on synchronization become worth the operational overhead.

19. Data Model at a Glance

```
User
|
+-- EmailAccounts
|
+-- Companies
|
+-- Contacts
|
+-- JobPostings
|
+-- JobApplications
|
+-- JobEvents
+-- JobEventCandidates
+-- ApplicationTasks
+-- ApplicationDocuments
+-- ApplicationContacts
+-- Interviews
+-- ExternalMessages

System:
GmailSyncState
AIRequestLog
OutboxEvent
```

20. Design Principles to Keep

Optimize away bookkeeping: The product wins when job-search state maintains itself.

Preserve source material: Retain the original JD and submitted documents, not only extracted fields.

Events over mutable history: Use JobEvent as the audit trail and derive coarse state from it.

AI proposes; domain decides: Keep classification probabilistic and mutations deterministic.

Review ambiguity: Low-confidence entity matching should become an explicit review task.

Modular monolith first: Keep deployment and debugging simple until scale demands separation.

Build attention before chat: A good morning attention view is more useful than a generic chatbot.

Generalize integrations at the edge: Normalize Gmail/Outlook into ExternalMessage rather than polluting the domain.